

설계·이해와 병렬 워크플로

글로 정의하고, 남의 코드를 읽고, **병렬로 부러라** — 명세가 명확할수록 **위임**할 수 있다



자원



스펙



설계문서



기존코드



검증



Worktree



Subagent



MCP-DB

대상 Day 1 수료(harness.git 안전망·Plan/권한) **권장** 8교시(교시당 50분) · 하루 8시간

이 하루가 답하려는 질문 – 무엇을 만들지가 흐리면 좋은 코드는 안 나온다



질문

Day 1에서 환경(harness)은 갖췄다. 하지만 –

무엇을 만들지가 흐리면 좋은 코드는 안 나오고,
실무 코드는 백지가 아니라 남이 짜 둔 것이며,
한 번에 하나만 부리면 사람이 병목이 된다.

오늘 얻을 세 능력 – **글로 정의** · **남의 코드 이해** · **병렬 위임**.



세 능력 = 오늘의 뼈대

- ① **정방향** 무엇을 만들지 글로 – 스펙·설계·테스트가 AI의 지시서
- ② **역방향** 남의 기존 코드를 Explore→Plan→Code→Commit으로 파악
- ③ **병렬** Worktree·Subagents·Agent Teams로 나눠 맡기고, MCP로 외부(DB)까지



오늘의 메시지 "글로 정의하고, 남의 코드를 읽고, 병렬로 부려라 – 명세가 명확할수록 나눠 맡길 수 있다."

학습 목표 – 오늘 끝나면 할 수 있는 것

1

컨텍스트·비용을 자원으로 — /cost·compact, 모델 티어링·effort, CLAUDE.md 다이어트

1교시

2

막연한 지시 vs 스펙 주도를 실측하고 PRD→태스크 분해→구현 지시 파이프라인

2교시

3

5대 설계문서(유스케이스~ERD)를 AI 초안+사람 검수 + Mermaid + 스캐폴딩

3교시

4

기존 코드베이스를 Explore→Plan→Code→Commit — 특성화 테스트로 안전 리팩토링

4교시

5

테스트 주도 vibe coding — RED→GREEN, cheating 방지, 혹 게이트, 시각 피드백

5교시

6

Git Worktree로 여러 Claude 병렬 실행 + 충돌 없는 분할·병합 패턴

6교시

7

Subagents를 .claude/agents/로 정의 + Agent Teams — 병렬 3축을 구분

7교시

8

MCP로 PostgreSQL 연결 — ERD를 실제 스키마로, 자연어로 쿼리·마이그레이션

8교시

오늘의 사고틀 — 설계·이해 축과 병렬 3축

두 축이 교차한다 — 전반(1~5)은 '무엇을 만들지'(설계)와 '이미 있는 게 뭔지'(이해)를 글로 잡고, 후반(6~8)은 나눠 맡기고 외부와 잇는다.

설계·이해·검증 축 · 전반 (2~5교시)

2 스펙 — 좋은 지시 → PRD → 분해 → 구현 지시

3 설계문서 — 5대(유스케이스~ERD) + 스캐폴딩

4 기존 코드 — Explore→Plan→Code→Commit

5 검증 루프 — 테스트 먼저 → 게이트 + 시각 피드백

1교시 · 컨텍스트·비용 관리 — 두 축의 공통 기초 (병렬 = 컨텍스트 × N · 비용 × N)

병렬·연결 축 · 후반 (6~8교시)

6·7 병렬 3축 — Worktree / Subagents / Agent Teams

8 연결 — MCP + PostgreSQL: ERD가 실제 DB가 된다

전제 조건 전반부가 후반부의 입력 — 계약·경계가 있어야 병렬이 충돌 지옥이 아니라 분업이 된다.

공통 원리 명세가 명확할수록 위임할 수 있고, 위임할수록 명세가 중요해진다 — 정방향(스펙→설계)·역방향(기존 코드)·검증이 갖춰져야 후반의 병렬이 가능해진다.

교시별 구성 – 8교시 한눈에 (Day2/3)

교시	등급	주제	형태	소주제	핵심 산출물
1	★★	컨텍스트·비용 관리	실습	9	/cost 실측 + 다이어트된 CLAUDE.md
2	★★	스펙 주도 개발	실습	9	관통 프로젝트 PRD + 태스크 분해표
3	★★	5대 설계문서 (유스케이스~ERD)	실습	12	설계문서 5종 + Mermaid + 스캐폴딩
4	★★	 기존 코드베이스 (E→P→C→C)	실습	10	온보딩 + 특성화 테스트 + 안전 리팩토링
5	★★	검증 루프 – 테스트 주도	실습	10	RED→GREEN + 게이트 훅 + 시각 피드백
6	★★★★	Git Worktree 병렬 세션	실습	9	2-worktree 병렬 구현 + 병합
7	★★★★	Subagents & Agent Teams	실습	10	커스텀 서브에이전트 + 병렬 3축
8	★★	MCP 연동 + DB	실습+ipynb	9	PostgreSQL 연결 + ERD→스키마·쿼리

이론:실습 ≈ 3:7 ★★★★★=필수 핵심(6·7 병렬) · ★★=심화. 3교시(12소주제)가 가장 밀도가 높다 – Mermaid는 렌더로 확인, 스캐폴딩까지 완주.

실습 방식과 이미지 표기 규칙



실습 방식 — 인라인(지시서 없음)

Day 2도 대부분 **터미널에서 AI를 조종하고 파일을 만드는** 형태 → 각 교시 [실습] 절에 인라인.

① 데모(강사) → ② 따라하기 → ③ 관통 프로젝트 적용

프롬프트 예시 · 체크포인트 포함.

예외 — 8교시 DB는 보조 노트북 `day2/08_postgres_db.ipynb`(MCP가 해 주는 일의 원리를 직접).



이미지 표기 규칙 (🖼️)

[로고]

제품·프로젝트 로고

[스크린샷]

실제 터미널·화면 캡처 (강사 삽입)

[다이어그램]

개념도 — 도형/Mermaid로 제작

[사진/그림]

개념 일러스트



점선 플레이스홀더 🖼️ 자리는 점선 박스로 잡아 두고 실제 화면은 강사가 삽입한다. [다이어그램]은 이 텍에서 도형으로 직접 그린다. 전체 목록은 부록 D(29점).



PERIOD 1 · 실습 ★★

컨텍스트·비용 관리

후반(6~7교시)에 에이전트를 여러 개 돌린다 — 컨텍스트도 비용도 $\times N$. 하나를 아껴 쓰는 법을 모르면 병렬은 '낭비의 병렬'이 된다. 게다가 컨텍스트는 길어질수록 품질이 떨어진다(앞의 결정을 잊는다). 자원 관리 = 비용 절약 + 품질 유지 — 오늘의 공통 기초.

왜 자원 관리부터인가 + 컨텍스트 윈도우의 실체

▲ 넘치기 직전



가장 빨리
쌓인다



작업 기억에 쌓이는 것

컨텍스트 윈도우 = [시스템 프롬프트] + [설정 파일] + [대화 이력] + [도구 결과].

파일 하나 읽으면 그 내용 전체가, 테스트를 돌리면 그 출력이 들어간다 — **도구 결과가 가장 빨리 쌓인다.**

긴 탐색 몇 번이면 윈도우가 가득 찬다. 그래서 '무엇을 읽게 할지'도 설계의 일부다.



[다이어그램] 왼쪽 컵이 그 적층 — 시스템/설정/대화/도구 결과. 자원 관리의 돈 문제이자 품질 변수다(길수록 초점이 흐려진다).

쓰는 만큼 보인다 + /clear vs /compact



[스크린샷] /cost 출력 (1.3)

세션 토큰 사용량·비용이 표시된 화면. 긴 탐색·큰 파일 읽기 후 확인. "감이 아니라 숫자로"

/clear vs /compact — 초기화와 압축

/clear

대화를 완전 초기화 (설정 파일은 다시 로드)

언제 — 작업 단위가 끝났을 때 · 새 주제 시작

/compact

대화를 요약해 압축 후 계속

언제 — 같은 작업을 이어가는데 컨텍스트가 부담될 때

주의 /compact는 요약 → 세부 소실 가능. 중요한 결정은 압축 전에 CLAUDE.md·메모리로 옮겨 둔다.



요령 긴 대화 하나보다 짧은 세션 여러 개 + 기억 파일이 낫다 — 필요한 맥락은 코드·문서·메모리에 있어야 정상.

세션 전략 + 모델 티어링 – 작업에 맞게

작업 단위로 끊는 리듬 – 깨끗한 컨텍스트로 시작하면 품질이 일정하고 비용도 예측 가능하다





세션 전략 – 작업 단위로 끊어라

컨텍스트는 날아가도 **기억 파일은 남는다**(Day 1 MEMORY.md).

"대화가 산으로 갔다" 싶으면 미련 없이 /clear – 필요한 맥락은 파일(코드·문서·메모리)에 있어야 정상이다.

모델 티어링 – /model 로 전환

최상위 · Opus급	설계·아키텍처·어려운 디버깅·긴 계획
중간 · Sonnet급	일반 구현·리팩토링 – 기본값
요령 계획은 비싼 모델로, 실행은 싼 모델로 (Plan Mode 조합).	



핵심

모든 작업에 최고 모델은 낭비 – 작업 성격에 맞춰 티어를 고른다. 상위 모델로 계획, 승인 후 하위 모델로 실행이 실무적.

effort 레벨 + CLAUDE.md 다이어트 – 고정비를 줄여라



effort – 같은 모델, 다른 깊이

모델 교체 없이 **추론 깊이(effort)**를 조절 – 높이면 더 오래 생각(품질↑, 비용·지연↑), 낮추면 빠르고 싸다.

조절 축이 두 개 – 티어링과 조합:

- 단순 작업 = 경량 모델 + 낮은 effort
- 핵심 설계 = 상위 모델 + 높은 effort



CLAUDE.md 다이어트 – 매 세션 고정비

빨 것 코드를 보면 아는 것 – 폴더 구조 나열, 자명한 컨벤션, 한 번 쓴 히스토리, 장황한 배경.

남길 것 코드를 봐도 모르는 것 – 하지 말 것, 함정, 실행·테스트 방법, 프로젝트 특수 규칙.



요령 AI에게 시킨다 – "CLAUDE.md에서 코드만 봐도 알 수 있는 내용을 찾아 삭제안을 제안해줘." 길수록·잡음 많을수록 정작 중요한 규칙이 묻힌다.

[실습] 비용 실측 + 다이어트 – 1교시 정리

> 관통 프로젝트에서 실측

```
# 실측① 압축 효과를 숫자로
> 저장소 전체 구조를 분석해줘
  /cost    → 토큰·비용 확인
  /compact → /cost 다시 (전후 비교)

# 실측② 같은 단순 작업(주석 추가)
  상위 모델 vs 경량 모델 – 속도·비용 체감
# 다이어트: CLAUDE.md 삭제안 제안받기
→ 짧아진 CLAUDE.md 커밋 ✓
```

체크포인트

- ☐ /cost 숫자를 읽을 수 있는가
- ☐ compact 후 세부 소실을 확인했는가
- ☐ CLAUDE.md가 실제로 짧아졌는가



1교시 정리 컨텍스트는 유한한 자원이자 품질 변수 – /cost로 보고, /clear-/compact로 관리, 작업 단위 세션+기억, 티어링·effort로 깊이 선택, CLAUDE.md 다이어트. 다음(2교시): 무엇을 만들지를 글로 못 박는다.



PERIOD 2 · 실습 ★★

스펙 주도 개발

"로그인 기능 만들어줘"는 열 번 시키면 열 가지가 나온다 — AI가 못해서가 아니라 정보가 없어서다. Day 1이 환경 (harness)을 갖췄다면 오늘은 지시의 내용을 갖춘다. 잘하는 사람일수록 더 명확하게 쓴다 — 'vibe'가 대충이 아니라는 증명.

왜 스펙인가 + 좋은 지시의 4요소



① 의도 (무엇을)

만들 것을 한 문장으로 — "이메일/비밀번호 로그인 API"



② 제약 (어떻게·금지)

스택·패턴·금지 — "FastAPI·bcrypt, 평문 저장 금지, User 재사용"



③ 컨텍스트 (어디에)

관련 파일·기존 구조 — "auth/ 아래, db.py의 세션 사용"



④ 성공 기준 (어떻게 확인)

검증 방법 — "틀린 비번은 401, 성공은 JWT 반환. 테스트 포함"



💡 관점 전환 "정답이 뭐지?"가 아니라 "이 결과가 만족해야 할 조건이 뭐지?"를 담아라 — 조건이 명확할수록 재현 가능해지고, 5교시에서 이 '성공 기준'이 그대로 테스트 코드가 된다.

좋은 vs 나쁜 지시 실측 – 명확함이 재현성을 만든다

항목	막연	4요소
되문기·추가 지시	여러 번	0~1
에러 처리 포함	제각각	✓
테스트 포함	X	✓
그대로 사용 가능	X	✓



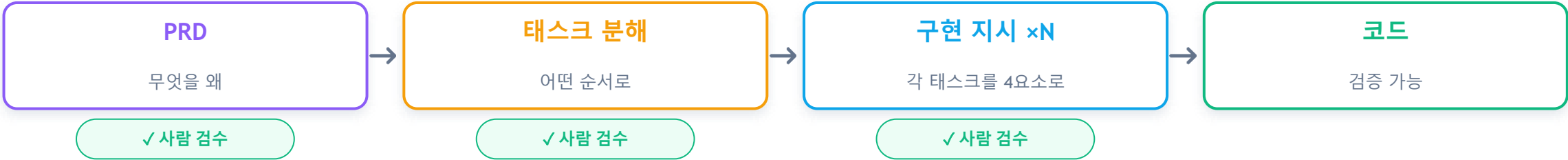
[스크린샷] 좋은 vs 나쁜 지시 결과 (2.3)

같은 요청의 두 결과를 좌(막연: 스택 제각각)/우(4요소: 요구 충족)로 나란히. "명확함이 재현성을 만든다"



체크포인트 "config.yaml을 읽어 dict로 반환하는 load_config(path). 파일 없으면 FileNotFoundError, pytest 포함" — 두 결과의 차이를 4요소 관점에서 설명할 수 있는가.

지시에서 스펙으로 – 파이프라인 + PRD



PRD (Product Requirements Document) – 관통 프로젝트 뼈대		항목
문제	흩어진 공개 데이터를 매번 수동으로 확인해야 한다	
사용자	특정 주제의 최신 동향을 요약으로 받고 싶은 개인/팀	
핵심 기능	수집(크롤링) → 저장(DB) → 요약(LLM) → 알림(Discord)	
비기능 요구	하루 1회 자동 실행, 실패 시 재시도, 시크릿은 .env	
성공 지표	매일 아침 요약 알림 도착률	

 PRD는 '왜' 구현 세부(어떤 라이브러리)는 아직 없다 – 그건 설계문서(3교시)의 일. 각 단계마다 '사람 검수'가 붙는다.

태스크 분해 – 검증 가능한 단위로

PRD를 실행 가능한 단계로 쪼갬다. 기준은 하나 – 각 단계에 검증 방법이 있는가.

단계	무엇을	검증 방법
①	데이터 모델(Item) 정의	필드가 유스케이스를 커버하나
②	크롤러 1개 소스	실제 몇 건 가져오나
③	DB 적재	저장·조회 되나
④	요약 함수	입력 → 요약 나오나
⑤	Discord 알림	메시지 도착하나
⑥	파이프라인 연결	end-to-end 도나



분해의 신호 검증 방법이 안 떠오르는 단계는 너무 크다는 신호 – 더 쪼갬다. 이 분해가 6교시(Worktree 병렬)에서 "누구에게 뭘 맡길지"의 단위가 된다.

태스크 → 구현 지시 – 파이프라인 완성

</> 태스크 ① → 4요소 지시 (프롬프트)

Item 데이터 모델을 만들어줘.

- 필드: id, source, url, title, content, created_at
(PRD의 수집 유스케이스 기준)
- dataclass로, store/models.py 에
- 성공 기준: 샘플 데이터로 생성/직렬화 되는 pytest 테스트 포함

세 층이 갖춰지면 지시는 반복 가능

PRD = 왜

태스크 분해 = 순서

4요소 지시 = how-to-verify

PRD가 '왜', 태스크가 '순서', 4요소 지시가 'how-to-verify' — 누가 시켜도 비슷한 결과.



2교시 정리 막연한 지시는 막연한 코드 — 의도·제약·컨텍스트·성공 기준 4요소로 쓰고, 규모에선 PRD→태스크 분해→구현 지시로 문서화. 분해 기준=검증 가능성, 이 분해가 병렬 분할 단위. 다음(3교시): 시스템의 모양=설계문서.



PERIOD 3 · 실습 ★★

5대 설계문서 — 유스케이스~ERD

PRD는 '무엇을 왜'까지만 말한다. 시스템의 모양(무슨 부품·어떤 계약·어떤 데이터)은 설계문서가 정한다 — 사람에게겐 합의의 기록, AI에겐 작업 지시서. 설계가 틀리면 이후 모든 코드가 틀리므로 발산은 AI·수렴은 사람으로 다섯 장을 만든다.

왜 설계문서인가 + 서로 물리는 다섯 장



발산은 AI 빠짐없이 나열(초안·예외 케이스 브레인스토밍)

수렴은 사람 우리 상황에 맞는 선택(규모·트레이드오프·우선순위)

반드시 이 순서 ①이 ②를, ②가 ③을, ③이 ④를 놓고, ①·③이 ⑤를 규정한다
— 앞 문서가 뒤 문서의 입력이며, 각 문서는 코드 산출물로 내려간다.

 **검수의 경제학** 검수 없이 넘기면 그럴듯한 오답 위에 집을 짓는다 — 설계 단계의 오류는 코드 단계 수정의 수십 배 비용. 30초 검수가 가장 수익률 높은 투자다.

유스케이스 명세서 – 누가, 무엇을, 왜

</> docs/유스케이스.md – 한 UC의 표준 구조

UC-01: 매일 아침 요약 받기

- 액터: 구독자
- 목적: 손대지 않아도 관심 주제 요약이 도착
- 사전조건: 수집 소스가 1개 이상 등록됨
- 기본 흐름: 스케줄러 → 신규 수집
→ 요약 생성 → Discord 발송
- 예외 흐름: 수집 실패 시 재시도,
신규 없음 시 "새 소식 없음" 발송



예외 흐름이 품질을 가른다

정상 경로는 누구나 쓴다 — 실패·빈 결과·중복을 적어야 비로소 '스펙'이 된다.

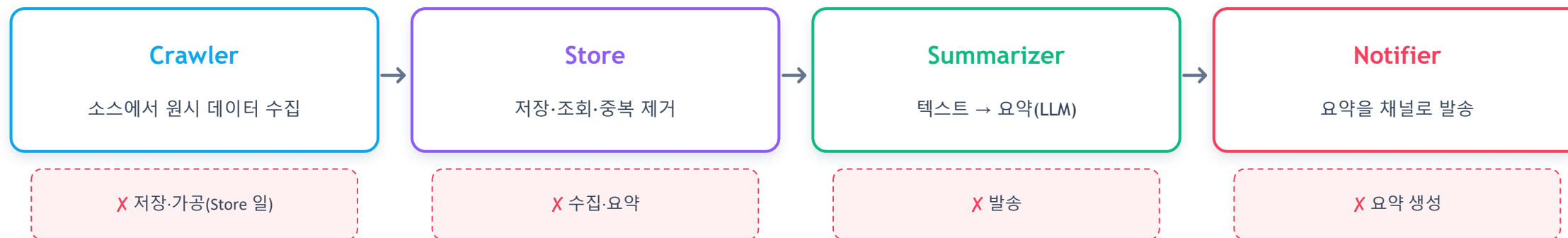
[실습] "docs/PRD.md 를 읽고 핵심 유스케이스 3~5개를 UC 형식으로 작성해줘."

검수 포인트 빠진 액터(관리자·시스템 자신)? 예외 흐름이 현실적? PRD 밖 기능이 끼어들지 않았나(스코프 크립)?



[스크린샷] AI가 PRD를 읽고 UC 3~5개를 뽑아낸 화면(검수 코멘트 포함). "AI 초안 + 사람 검수 — 검수 없는 초안은 소설이다." 저장·커밋: docs/유스케이스.md.

컴포넌트 설계서 – 부품과 '하지 않는 것'



[실습] "docs/유스케이스.md 기준 컴포넌트 설계서 – 책임/하지 않는 것/의존 방향. 4±1개로." **검수** 과하게 많지 않나(마이크로서비스 12개?) · 의존이 한 방향인가(순환 금지) · 유스케이스가 전부 커버되나

💡 경계 = 병렬의 전제 핵심은 '하지 않는 것' 열이다 — 경계가 명확해야 6교시에서 컴포넌트 단위로 병렬 분업이 가능하다(흐리면 두 에이전트가 같은 파일을 고친다).

인터페이스 정의서 – 시그니처가 곧 계약

</> docs/인터페이스정의서.md – 함수 계약

```
# Crawler
def crawl(source: Source) -> list[Item]: ...

# Store
def save_items(items) -> int: ... # 신규 건수
def get_unsummarized() -> list[Item]: ...

# Summarizer
def summarize(items) -> str: ...

# Notifier
def notify(message: str) -> bool: ... # 발송 여부
```



계약이 고정되면 구현을 서로 몰라도 된다

A가 `crawl()`을 어떻게 짜든 `save_items()`는 `list[Item]`만 받으면 된다 — 6교시 병렬의 기술적 토대(계약 먼저, 구현은 각자).

[실습] "컴포넌트설계서 기준으로 인터페이스를 파이썬 시그니처+한 줄 docstring으로. 반환 타입과 실패 시 동작을 명시."

검수 실패 시 동작이 계약에 있나(예외? None? False? — 애매하면 구현자마다 달라진다).

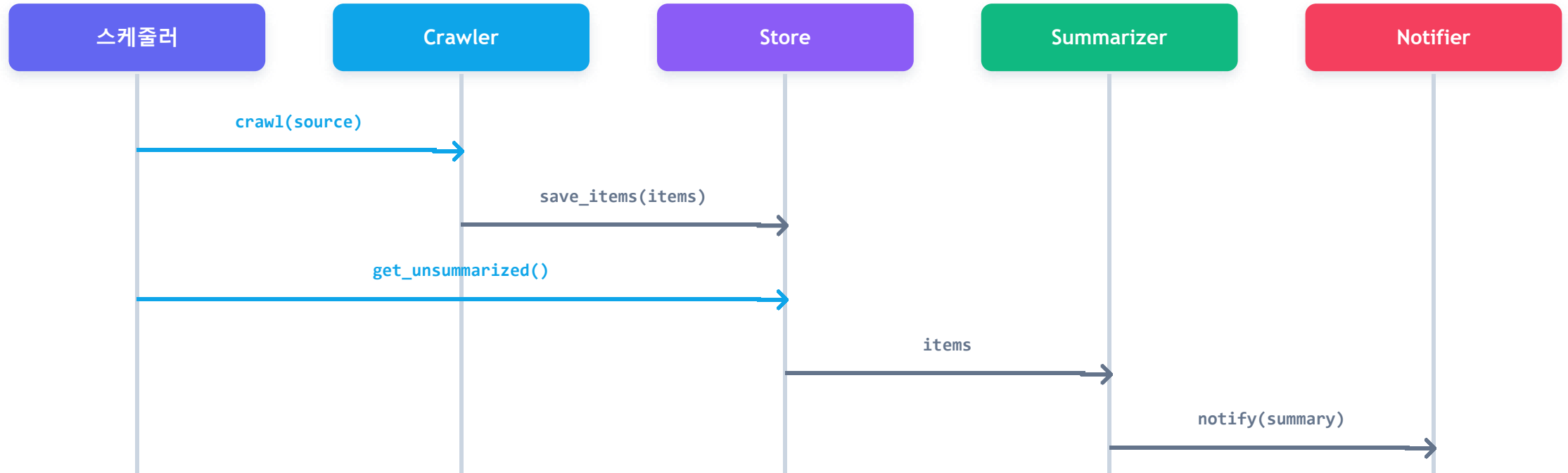


계약 = 시그니처 + 실패 동작 반환 타입만이 아니라 '실패하면 어떻게 되는가'까지 못 박아야 계약이다. 유스케이스의 예외 흐름을 지원하는지 확인. 저장 : docs/인터페이스정의서.md.

3.8 SEQUENCE

시퀀스 다이어그램 – 시간 순 상호작용

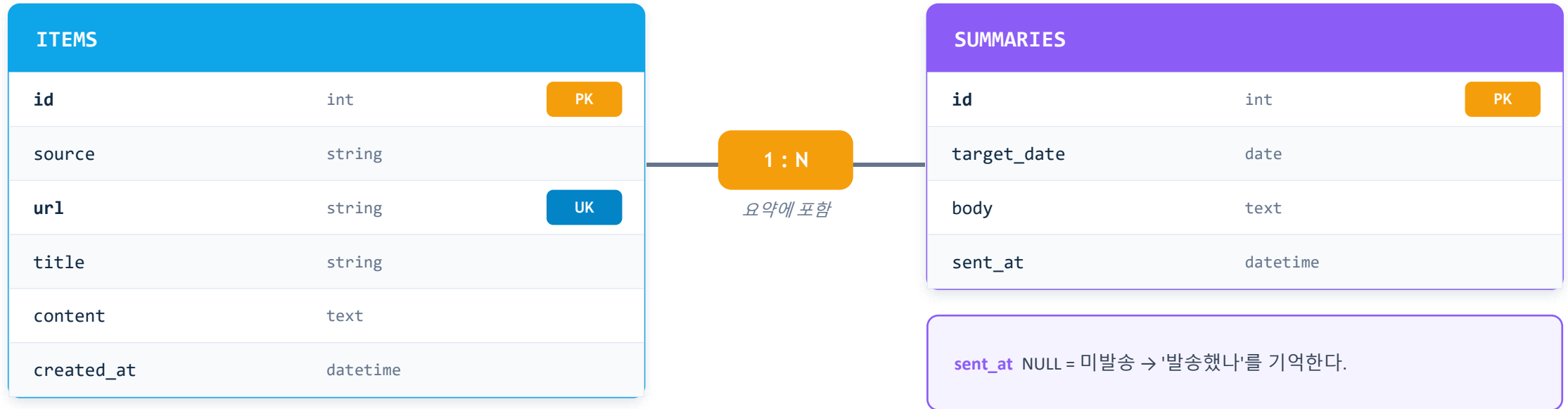
누가 누구를 어떤 순서로 부르나 – Mermaid로 쓰면 텍스트라 AI가 생성·수정하고 git 이력도 남는다.



 **[다이어그램]** 위 도형이 본문 Mermaid sequenceDiagram의 렌더 — 스케줄러가 Crawler·Store·Summarizer·Notifier를 시간 순으로 호출. 생성물은 뷰어·에디터 프리뷰로 렌더해 눈으로 확인한다. 저장: docs/시퀀스.md.

ERD – 데이터의 모양 (칼럼 = 유스케이스 문장)

유스케이스('무엇을 기억하나')와 인터페이스('무엇이 오가나')에서 테이블이 나온다.



칼럼 = 유스케이스 문장 url UNIQUE가 중복 수집을 막고(예외 흐름!), sent_at이 발송을 기억한다. 이 ERD는 8교시에서 실제 PostgreSQL 스키마가 된다. 저장: docs/ERD.md.

일관성 교차 검수 – 모순은 코드 전이 가장 싸다

점검	질문
인터페이스 ↔ 컴포넌트	모든 컴포넌트에 함수 계약이 있나?
시퀀스 ↔ 인터페이스	시퀀스의 호출이 전부 인터페이스에 정의됐나?
ERD ↔ 유스케이스	유스케이스가 필요로 하는 데이터가 다 있나? (중복 방지? 발송 기록?)

 **검수도 AI에게** "docs/ 의 설계문서 5종이 서로 모순되는 부분을 교차 점검하고 수정안을 제안해줘." — 단, 최종 판단은 사람(3.1 원칙 그대로).

 **다섯 장이 서로 물린다** 모순은 코드가 되기 전이 가장 싸다 — 세 축으로 교차 점검한 뒤에야 스캐폴딩으로 내려간다.

문서 → 코드 – spec-driven 스캐폴딩

</> > 문서를 작업 지시서로 (스캐폴딩 프롬프트)

> docs/ 의 설계문서대로 스캐폴딩을 만들어줘:

- ① 컴포넌트설계서 → crawler/ store/
 summarizer/ notifier/ + main.py
 - ② 인터페이스정의서 → 각 모듈 함수 스텝
 (구현은 raise NotImplementedError)
 - ③ ERD → store/schema.sql
- 실제 로직은 만들지 말고 구조만.
끝나면 tree 로 구조를 보여줘.

문서 → 코드 매핑

컴포넌트 설계서

→ 폴더 / 패키지 구조

인터페이스 정의서

→ 함수 스텝 (NotImplementedError)

ERD

→ store/schema.sql (CREATE TABLE)

구현은 아직 없다 – 뼈대만 (구현은 5·6교시)



3교시 정리 다섯 장이 순서대로 물린다 – 유스케이스(예외=품질)·컴포넌트(경계)·인터페이스(계약)·시퀀스·ERD를 AI 초안+검수, 교차 검수 후 스캐폴딩 까지. git commit "Day2: 설계문서 5종 + 스캐폴딩". 다음(4교시): 남의 기존 코드 이해.

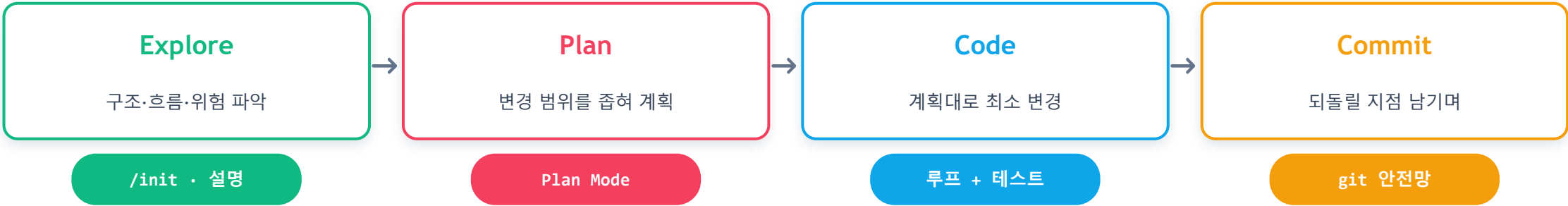


PERIOD 4 · 실습 ★★ · 

기존 코드베이스 — Explore→Plan→Code→Commit

실무의 대부분은 백지가 아니라 남이 짜 둔, 문서도 부족한 기존 코드다. "여기 로그인 버그 고쳐줘"가 어려운 건 AI도 나도 이 코드가 뭘 하는지 모른 채 손대기 때문 — 3교시가 정방향(무엇을 만들지)이라면 이 교시는 역방향(이미 있는 것이 무엇인지)이다.

Explore→Plan→Code→Commit – 낯선 코드의 4박자

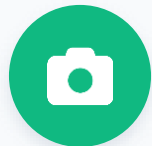


↺ Commit 후 다음 변경은 다시 Explore부터 – 낯선 코드일수록 앞(Explore·Plan)에 무게가 실린다

💡 **화살표가 반대다** 그린필드에선 설계문서가 코드를 낳지만, 레거시에선 코드에서 이해를 길어 올려 머릿속 지도를 만든다. 그래서 첫 수가 '구현'이 아니라 '탐색'이다.

💡 **Code는 4박자 중 하나** v3 조각(`/init·Plan·git`)을 하나의 루프로 꿰고, 레거시 특유의 위험(모르고 부수기)을 다루는 법을 더한다.

Explore – 저장소에 말 걸기 + 위험지점 지도



[스크린샷] /init · 구조 설명 (4.3)

낯선 저장소에서 /init 후 "핵심 흐름 설명해줘"에 진입점 → 모듈 흐름을 짚어 주는 화면. "손대기 전에 지도부터"



① 저장소에 말 걸기

/init 로 CLAUDE.md 초안(스택·구조·실행법) → 없던 지도. 이어서 Q&A — "핵심 흐름을 진입점부터", "이 기능은 어느 파일?", "테스트는 어떻게 돌리나?"



② 위험지점·의존성 지도

"이 함수를 바꾸면 영향받는 곳을 전부(호출부·테스트·설정)", "외부 의존성(DB·API·전역·파일 IO)은 어디?", "테스트가 없는 위험한 부분은?"



읽기 위주 이 단계에선 수정 권한이 필요 없다(Plan/Ask로 충분) — 탐색의 목적은 지도를 그리는 것이지 고치는 게 아니다. 모르는 코드일수록 범위를 좁게.

Plan(범위 좁히기) + 특성화 테스트



Plan – 변경 범위를 좁히기

탐색으로 지도가 생겼으면 Plan Mode로. 확인할 것:

- 변경이 최소 범위인가 (스코프 크립?)
- 위험지점을 건드리나? 어떻게 보호하나
- 되돌릴 수 있나 (작은 커밋 단위)

계획 30초가 잘못된 구현 30분을 아낀다 — 남의 코드에선 배율이 더 크다.



특성화 테스트 – 현재 동작을 고정

레거시엔 스펙 문서가 없다. '옳은지'는 몰라도 '지금 이렇게 동작한다'는 사실 — 이 현재 동작을 붙잡는 테스트.

① 현재 입출력 관찰

② 그대로 assert

③ GREEN 확인

④ 안심하고 리팩토링

⑤ 깨지면 = 동작이 바뀐 것



양방향 안전망 그린필드는 테스트→구현, 레거시는 구현→테스트로 감싸기 — 5교시 검증 루프를 기존 코드에 두르는 응용. "이 함수의 현재 동작을 특성화 테스트로 감싸줘."

[실습] 특성화 테스트로 감싸고 안전하게 리팩토링

> 초록을 유지하는 한 겁 없이 고친다

```
# ① 현재 동작 고정 (구현은 건드리지 마)
> 이 함수의 현재 동작을 특성화 테스트로
$ pytest tests/test_util.py
=== 3 passed (초록) ===
# ② 초록 유지하며 읽기 쉽게 리팩토링
(Plan Mode로 계획부터) 동작은 그대로
$ pytest → === 3 passed ===
# 빨강이면? 동작이 바뀐 것 → 되돌리고
다시 (Day 1 4교시 restore)
```

체크포인트

- ☐ 특성화 테스트가 리팩토링 전/후 모두 초록인가
- ☐ 일부러 동작을 바꾸면 빨강으로 잡히는가
- ☐ 코드를 손대기 전에 Explore·Plan을 실제로 거쳤는가



 [스크린샷] 리팩토링 전후로 특성화 테스트가 초록 유지되는 pytest 화면(리팩토링 diff와 나란히). "초록을 유지하는 한, 겁 없이 고친다."

체계적 디버깅 – 증상에서 원인으로

단계	하는 일	AI 활용
① 재현	버그를 항상 터지게 하는 최소 입력	"이 버그를 재현하는 최소 케이스를 만들어줘"
② 가설	"원인은 x일 것" 후보를 좁힌다	"이 증상의 원인 가설을 3개, 가능성 순으로"
③ 최소 실험	가설 하나만 검증하는 작은 변경.로그	"가설 ①만 확인할 로그/실험을 제안해줘"
④ 검증	실험 결과로 채택/기각 → 반복	재현 케이스가 통과하면 수정 완료


⚠ 재현 케이스를 먼저 AI에게 원인 규명을 통째로 맡기면 그럴듯한 오진(Day 1 1.5). 재현(①)을 먼저 두면 어떤 수정이 진짜 고친 건지 기계가 판정한다 – 증명은 테스트.


4교시 정리 첫 手는 Explore(이해) – /init:Q&A로 지도, Plan으로 범위, 특성화 테스트로 현재 동작 고정 후 안전 변경, 되돌릴 지점(Commit). 다음(5교시): 새 로 만들 동작을 테스트로 먼저 정의.

5

PERIOD 5 · 실습 ★★

검증 루프 — 테스트 주도 Vibe Coding

AI 코드는 그럴듯하게 틀린다(Day 1 1.5). 내가 안 짤 코드를 믿는 방법은 하나 — 기계가 검증하게 만드는 것. 눈으로 훑는 리뷰는 사람이 병목이고(병렬이면 $\times N$), "실행해 보니 되던데"는 한 경로만 본 것이다. 테스트는 반복 가능하고 자동이며 병렬 결과물에도 똑같이 적용된다.

테스트가 곧 명세다 + RED→GREEN 리듬

① 실패하는 테스트 (RED) 구현이 없으니 당연히 실패



② "이 테스트를 통과시켜줘" AI가 구현



③ 실행 → GREEN 확인 통과



④ (필요 시) 리팩토링 테스트가 있으니 겁 없이

</> 테스트 = 실행 가능한 스펙 (test_store.py)

```
def test_save_items_skips_duplicate_url():  
    n1 = save_items([item_a, item_b]) # 2건  
    n2 = save_items([item_a]) # 같은 url  
    assert n1 == 2 and n2 == 0  
    # 중복 저장 안 됨 ← ERD의 url UNIQUE
```

"중복 처리 잘 해줘"(모호) → "이 테스트를 통과시켜줘"(정확)

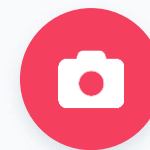


RED를 먼저 눈으로 실패하는 걸 본 적 없는 테스트는 '항상 통과하는 가짜'일 수 있다(assert가 안 도는 테스트, 잘못된 대상을 검증하는 테스트). 2교시의 '성공 기준'을 코드로 쓰면 그게 테스트다.

[실습] 실패하는 테스트 먼저 – Store 중복 방지

> 한 사이클 (3교시 스텝 → 구현)

```
# ① 테스트 작성 (구현은 하지 마)
> save_items 스펙(중복 제외, 신규 건수)
   테스트를 tests/test_store.py 에
# ② pytest → RED 확인
   E   NotImplementedError
# ③ 통과하도록 구현 (테스트 수정 금지)
# ④ pytest → GREEN 확인 → 커밋
   === 1 passed ===
→ diff 를 읽는다 (Day 1 4교시 습관)
```



[스크린샷] RED → GREEN (5.4)

같은 pytest 명령의 실행 전(FAILED 빨강)/후(passed 초록)를 위아래로. "실패를 먼저 봐야 테스트를 믿을 수 있다"



체크포인트 RED를 실제로 봤는가 / GREEN 후 diff를 읽었는가. "tests/test_store.py 를 통과하도록 구현해줘. 테스트 파일은 수정 금지."

Cheating 방지 + Hook 테스트 게이트 – 부탁을 강제로



AI는 '통과'를 최적화한다

가끔 문자 그대로 통과만 시킨다:

- 테스트 파일을 수정해 기준을 낮춘다 / skip
- 기대값을 하드코딩 (return 0 # 0을 기대하니)

방어 3종 ① "테스트 수정 금지" 명시 · ② 커밋 전 diff에서 tests/ 확인 · ③ 아예
혹으로 강제(→)

</> settings.json – PreToolUse(Bash) 게이트

```
# git commit 감지 → 먼저 pytest 실행
# 실패 시 exit 2 (커밋 차단 + 사유 전달)
"PreToolUse": [{ "matcher": "Bash",
  "hooks": [{ "command":
    "commit이면 pytest, 실패면 exit 2" }]}]
```

효과 cheating으로 로컬만 초록이어도, 빨간 상태로는 커밋 자체가 안 된다.



부탁 → 강제 "커밋 전에 테스트 돌려"는 부탁이다(Day 1 6교시: 부탁은 잊힌다). 혹으로 강제한다 — tests/ 수정을 PreToolUse(Edit|Write)로 차단하는 변형
도 가능.

[실습] 테스트 게이트 후 + 어디까지 테스트하나

> 일부러 깨뜨리고 커밋 시도

```
# ① 커밋 전 pytest 게이트 추가 (5.6)
# ② 일부러 테스트를 깨뜨리고 커밋
> git commit -m "wip"
• PreToolUse: pytest 실행...
❌ 1 failed → exit 2 (커밋 차단)
  사유가 에이전트에게 전달됨
# ③ 고치고 커밋 → 통과
=== passed === → 커밋 성공
(통과하면 게이트가 없는 것과 같다)
```



어디까지 테스트하나 – 우선순위

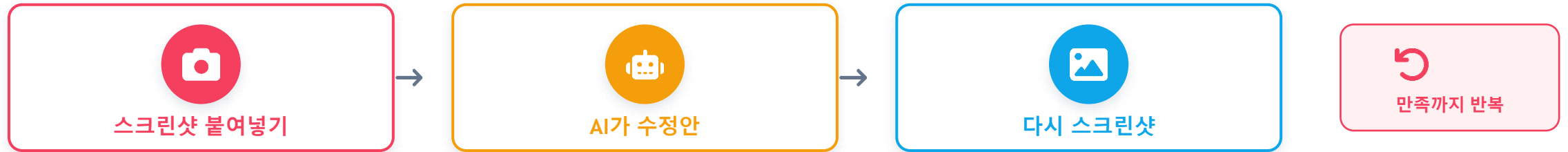
- ① 계약(인터페이스)의 성공/실패 동작
- ② 유스케이스의 예외 흐름
- ③ 과거에 터진 버그 (버그 = 새 테스트)

엣지 발굴은 AI에게 "save_items 엣지 케이스 5개(빈 리스트·필드 누락·같은 url 대량 ...) 각각 테스트로" — 발산은 AI, 우선순위는 사람.



체크포인트 깨진 상태의 커밋이 실제로 차단되는가 / 차단 사유가 에이전트에게 전달돼 스스로 고치는가 (Day 1 8.5 위반 테스트 원칙 — 일부러 어겨봐야 강제를 확인).

시각 피드백 루프 – 눈으로 보고 고치기



💡 두 피드백 루프는 짝 로직은 테스트(기계)로, 걸모습은 스크린샷(눈)으로 검증한다. RED→GREEN이 '숫자로' 도는 루프라면, 이건 '그림으로' 도는 루프 – 말로 어려운 '어색함'을 그림 한 장이 전달한다.



5교시 정리 테스트 = 실행 가능한 명세 – 성공 기준을 코드로, RED 확인 후 AI가 GREEN. cheating은 수정 금지+diff+혹 게이트로 막고, 눈에 보이는 건 시각 피드백으로. 다음(6교시): 이 기준 위에서 여러 에이전트를 동시에.



PERIOD 6 · 실습 ★★★

Git Worktree 병렬 세션

에이전트에게 큰 작업을 시키면 몇 분씩 걸린다 — 그동안 사람은 논다. 작업을 쪼개 여러 에이전트에 맡기면 대기가 겹쳐 사라진다: A를 검토하는 동안 B가 구현 중. 2교시의 태스크 분해와 3교시의 컴포넌트 경계가 있으면, 병렬은 자연스러운 다음 수다.

Worktree – 한 저장소, 여러 작업 폴더

브랜치가 여러 개라도 작업 폴더는 하나 – checkout 하면 이전 작업이 사라진다(stash 지옥). 두 브랜치를 동시에 열 수는 없나? 있다.



복제(clone)와 다르다 커밋·브랜치·이력이 실시간 공유되고(같은 .git) 디스크도 아낀다. 관례: worktree 폴더는 저장소 밖(형제 디렉토리)에 – 안에 만들면 서로를 오염시킨다.



하나의 .git을 공유하는 폴더 3개(브랜치별)가 동시에 존재 – "브랜치 전환 없이 동시에." 폴더마다 다른 브랜치를 체크아웃한 채 **동시에** 존재함.

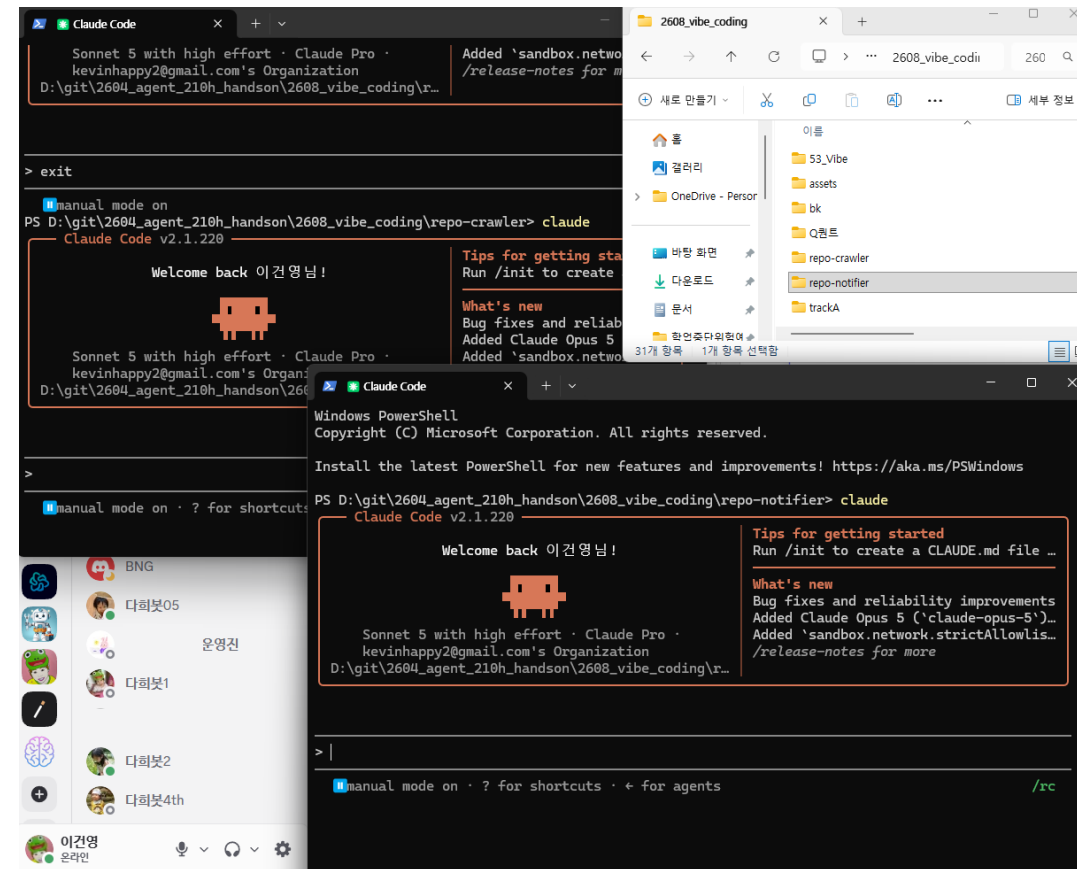
기본 명령 4개 + Worktree × Claude

</> 기본 명령 4 개 (bash)

```
git worktree add -b feat/crawler ../repo-crawler
git worktree add -b feat/notifier ../repo-notifier
git worktree list          # 현재 목록
git worktree remove ../repo-crawler
git worktree remove ../repo-notifier
git branch -d feat/crawler # 원복까지 하려면
```

각 worktree 폴더에서 **claude** 실행 → 완전 독립 세션.

CLAUDE.md·커맨드·혹(Day 1 harness)은 저장소 파일 → 모든 **worktree**에 그대로 적용.



터미널 분할 — 왼쪽 worktree에서 crawler, 오른쪽에서 notifier가 동시에 진행.
"검토하는 동안 다른 쪽이 구현한다"



harness의 보상 각 세션은 자기 브랜치의 파일·자기 컨텍스트를 가진다. harness를 잘 만들어 둔 보상이 병렬에서 돌아온다 — 규칙 재설명이 필요 없다.

작업 분할 원칙 + 충돌 방지 패턴

☑️ **좋은 분할** 컴포넌트 단위(crawler/ vs notifier/) — 3교시 경계('하지 않는 것')가 그대로 분할선.

❌ **나쁜 분할** 같은 모듈의 기능 A/B — 같은 파일을 양쪽에서 고쳐 병합 지옥. 공유 파일(models.py-인터페이스)은 병렬 전에 고정(3교시에서 이미!).

충돌 방지 패턴

계약 먼저	인터페이스·모델을 main에 먼저 커밋
폴더 소유권	"crawler/ 밖은 수정 금지" 명시
짧은 수명	worktree는 하루 이내로
자주 병합	끝난 것부터 바로 main에

💡 **부탁을 강제로** "수정 금지" 지시는 부탁이다 — 필요하면 흑으로 폴더 밖 수정을 차단(Day 1 흑 응용)해 강제할 수 있다.

계약이 main에 있는가 – 커밋된 것만 공유된다

</> ① Claude Code 에서

```
> crawler/ 와 notifier/ 패키지 스텝과 각각의
  테스트(tests/test_crawler.py,
  tests/test_notifier.py)를 만들어줘.
  구현은 NotImplementedError로 두고
  테스트는 "아직 실패하는 상태"로. 그리고 커밋해줘.
```

</> ② 일반 터미널에서 – 상태 확인

```
# (원본 저장소 폴더)
git status --short
git log --oneline -1
```

>_ 예상 결과 + .gitignore 확인

```
← git status가 아무것도 안 나옴
af3ac02 chore: 계약(스텝·테스트) 고정

# ⚠ .gitignore도 '커밋'돼 있어야 한다
$ git log --oneline -1 -- .gitignore
1a2b3c4 chore: .gitignore 추가

# 아무것도 안 나오면 아직 커밋 안 된 것 →
> .gitignore에 __pycache__/ 와 *.py[cod] 를
  넣고 커밋해줘.
```



왜 .gitignore를 먼저 worktree는 커밋된 것만 받는다 – 스테이징만 해 두면 worktree엔 .gitignore가 아예 없어서 __pycache__ 가 전부 딸려 커밋된다(6.9 ⑤).

worktree 두 개 만들기 + 공유 확인

</> 일반 터미널에서

```
# (원본 저장소 폴더)
git worktree add -b feat/crawler ../vibe-crawler
git worktree add -b feat/notifier ../vibe-notifier
git worktree list
cat ../vibe-crawler/.git
```

feat/ 는 git 문법이 아니라 팀 관례

Day 1의 커밋 컨벤션(feat/fix/docs...)을 브랜치 이름에도 쓴 것. my-branch 라 지어도 git은 똑같이 동작하지만, 병렬에선 이름만 보고 무슨 작업인지 알아야 한다.

>_ 예상 결과

```
Preparing worktree (new branch 'feat/crawler')
HEAD is now at af3ac02 chore: 계약 고정
Preparing worktree (new branch 'feat/notifier')
HEAD is now at af3ac02 chore: 계약 고정

D:/git/vibe_handson    af3ac02 [main]
D:/git/vibe-crawler    af3ac02 [feat/crawler]
D:/git/vibe-notifier    af3ac02 [feat/notifier]

gitdir: ../.git/worktrees/vibe-crawler
# .git이 '폴더'가 아니라 '파일' = clone이 아니다
```



복제가 아니라 작업 폴더 추가 저장소(.git)는 하나뿐이고 폴더만 늘어난 것 — 그래서 한쪽에서 커밋하면 다른 쪽에서 즉시 보인다.

폴더 두 종류를 구분하자 – 이름이 비슷해서 헷갈린다

</> D:/git/ 아래 실제 구조

```
D:/git/
├─ vibe_handson/      ← 원본 저장소 (main)      [worktree]
│   ├─ crawler/       소스 패키지              [소스 폴더]
│   ├─ notifier/      소스 패키지              [소스 폴더]
│   └─ tests/
├─ vibe-crawler/      ← worktree A (feat/crawler) [worktree]
│   ├─ crawler/       같은 소스 패키지 (이 브랜치 버전)
│   ├─ notifier/
│   └─ tests/
└─ vibe-notifier/     ← worktree B (feat/notifier) [worktree]
    └─ (구조 동일)
```



구분 vibe-crawler/(하이픈) = git이 만든 작업 폴더(저장소 전체가 그 브랜치 시점으로 들어 있다) · crawler/(안쪽) = 우리가 만드는 소스 패키지. 이름이 비슷한 건 우연 – 헷갈리면 ../wt-crawler 처럼 접두사를 달리해도 된다.

병렬 세션 + ⚠ 커밋 – 여기를 건너뛰면 (6)이 헛돈다

</> (3) 터미널 ②·③ 에서

```
# (worktree A)
cd ../vibe-crawler
claude
> tests/test_crawler.py 가 통과하도록
  crawler/ 를 구현해줘.
  crawler/ 밖의 파일은 수정하지 마.

# (worktree B)
cd ../vibe-notifier
claude
> tests/test_notifier.py 가 통과하도록
  notifier/ 를 구현해줘.
  notifier/ 밖의 파일은 수정하지 마.
```

</> (4) 각 worktree에서 커밋 – 필수

```
> 테스트 통과 확인하고 커밋해줘.

# 또는 직접 (worktree A 폴더)
git add -A
git commit -m "feat: crawler 구현"

# 확인
git status --short
git log --oneline -1
#   ← status는 비어 있어야 한다
#   f5be9c7 feat: crawler 구현

# ❌ ' M crawler/__init__.py' 가 남아 있으면 미커밋
```



이 교시에서 제일 중요한 한 문장 worktree는 '커밋'을 공유하지 '작업 중인 파일'을 공유하지 않는다 – 에이전트는 시키지 않으면 커밋하지 않으므로, 여기를 건너뛰면 (6)에서 'Already up to date.'가 나온다.

관찰 → 병합 → 정리

</> 일반 터미널에서

```
# (5) 관찰 - 원본 저장소 폴더에서
git log --oneline --all -3
git status --short

# (6) 병합
git merge feat/crawler

# 테스트 게이트 통과 확인 후 정리
git worktree remove ../vibe-crawler
git branch -d feat/crawler
git worktree list
```

>_ 예상 결과

```
f5be9c7 feat: crawler 구현
af3ac02 chore: 계약(스텝·테스트) 고정
      ← status는 비어 있음(내 폴더는 안 변함)

# 이력은 공유, 작업 폴더는 분리

Fast-forward
  crawler/__init__.py | 2 +-
  1 file changed, 1 insertion(+), 1 deletion(-)
Deleted branch feat/crawler (was f5be9c7).

D:/git/vibe_handson f5be9c7 [main]
```



Fast-forward가 곧 성적표 충돌이 났다면 도구가 아니라 분할이 나뉘었던 것(6.6) — 두 작업이 같은 파일을 건드렸다는 뜻이다. 정리 순서는 항상 merge → worktree remove → branch -d.

반대쪽도 완주 – 전체 흐름을 한 번에

</> 일반 터미널에서

```
# ① .gitignore 받아오기          ← 함정 ⑤ 예방
cd ../vibe-notifier
git merge main
# ② 커밋 전 눈으로 확인
git status --short
# ③ 커밋해야 공유된다          ← 함정 ④ 예방
git add -A
git commit -m "feat: notifier 구현"
# ④ 원본에서 병합 → 정리
cd ../vibe_handson
git merge feat/notifier
git worktree remove ../vibe-notifier
git branch -d feat/notifier
```

>_ 예상 결과

```
Updating 538105c..3cd3731
Fast-forward
.gitignore | 9 ++++++++

M notifier/__init__.py
# ?? __pycache__ 가 안 보이면 ⑤ 회피 성공

Fast-forward
Deleted branch feat/notifier (was ...).

D:/git/vibe_handson <해시> [main]
```



완주 신호 Fast-forward가 두 번 나오고 worktree 목록에 원본만 남으면 2-worktree 병렬 완주 – 분할(6.6)과 계약 고정(6.7)이 옳았다는 증거다.

실습 중 실제로 만나는 함정 5가지 (전부 실측)

⚠ **전제** worktree-브랜치가 아직 살아 있을 때 재현된다 — (6)에서 정리한 뒤 시도하면 조건이 사라져 엉뚱한 결과가 나온다(그냥 성공하거나 fatal: ambiguous argument). 정리 전에 해 보자.

#	증상	원인 · 대응
①	fatal: 'feat/x' is already used by worktree at ...	같은 브랜치를 두 폴더에서 못 연다. git branch -vv 의 '+' 로 사용 중 확인
②	fatal: ... contains modified or untracked files	미커밋 변경이 남은 worktree. --force 바로 쓰지 말고 커밋/폐기를 먼저 결정
③	worktree를 지워도 브랜치가 남는다	merge → worktree remove → branch -d 순서. -d 성공 = 병합 완료의 증거
④	Already up to date. (병합했는데)	worktree에서 커밋을 안 한 것. git log main..feat/x 로 확인(빈 출력=미커밋)
⑤	커밋에 __pycache__/*.pyc 가 딸려 갔다	.gitignore를 커밋 안 해 worktree에 없었던 것. git rm -r --cached 로 추적 해제

💡 **④와 ⑤는 같은 뿌리** worktree는 커밋된 것만 받는다 — 그래서 '커밋을 안 하면 안 넘어가고'(④), '.gitignore를 커밋 안 하면 worktree엔 없다'(⑤).

6교시 정리 – 성패는 도구가 아니라 분할이다



프로세스 수준 병렬

폴더·브랜치·세션이 완전히 분리된, 가장 안전한 병렬. 서로의 파일이 눈앞에 없으니 사고가 안 난다.



성패는 분할에서

컴포넌트 경계(3교시)를 분할선으로, 계약(인터페이스·모델)은 병렬 전에 main에 고정.



커밋이 공유의 단위

worktree는 커밋된 것만 주고받는다 – 커밋 누락·.gitignore 미커밋이 사고의 90%.



정리까지가 실습

merge → worktree remove → branch -d. 방치하면 브랜치 미로가 된다.



다음(7교시) 폴더를 나누지 않고 한 세션 안에서 위임하는 두 번째 병렬 – Subagents.



PERIOD 7 · 실습 ★★★

Subagents & Agent Teams

Subagent는 메인 세션이 하위 작업을 위임하는 별도 에이전트다 — 자기만의 컨텍스트 창에서 일하고 끝나면 결과 요약만 메인에 돌려준다. 팀장(메인)이 직접 다 하지 않고 팀원(서브)에게 "이거 조사해서 결론만 보고해"라고 시키는 그림. 그 위층이 Agent Teams다.

메인 → 서브 위임 – 독립 컨텍스트·독립 권한



🔄 끝나면 결과 요약만 메인에 반환

왜 위임하나 – 컨텍스트 절약 + 관점 분리

긴 탐색을 메인에서 하면 메인 컨텍스트가 오염된다(1교시 회수). 서브에게 시키면 탐색 부산물은 서브에서 소모되고 메인엔 결론만. 서브는 메인 대화에 물들지 않아 독립적 관점을 준다.

독립 컨텍스트 · 독립 권한

도구(권한)도 별도 제한 – 리뷰 서브엔 읽기만(Edit 금지), 테스트 러너 서브엔 Bash만. Day 1 최소 권한이 에이전트 단위로 확장 – "실수로 고치는 리뷰어"가 구조적으로 불가능해진다.



메인 세션에서 서브 2~3개로 위임(각자 독립 컨텍스트 상자), 돌아오는 화살표엔 '결과 요약만'. 탐색은 서브에서, 메인엔 결론만.

.claude/agents/ – 파일 하나로 정의한다

</> .claude/agents/code-reviewer.md

```
---
name: code-reviewer
description: 코드 변경을 리뷰할 때 사용.
  버그·보안·컨벤션 위반을 심각도순 보고.
tools: Read, Grep, Glob  # 읽기 전용
---

너는 시니어 리뷰어다. 심각도(높음/중간/낮음)
별로 보고하고, 문제 없으면 "이상 없음"이라 하라.
tools에 수정 도구가 없으니 제안까지만 하라.
```

프론트매터 3필드가 전부다

name 식별자(파일명과 맞춘다)

description 언제 이 서브를 쓸지 – 메인이 읽고 자동 위임(스킬과 같은 원리, Day 1 7.4)

tools 쓸 수 있는 도구 화이트리스트. **비우면 메인의 모든 도구 상속(위험 – 리뷰어가 코드를 고침)**



위치 .claude/agents/ (프로젝트 공유 – git 커밋) vs ~/.claude/agents/ (개인 – 모든 프로젝트). 커맨드·스킬과 같은 철학. 7.5에서 이 파일을 실제로 만들어 호출한다.

만들기 – AI에게 시키거나 이 블록을 그대로 붙여넣는다

</> ① claude 에서 AI에게 시키기

```
> .claude/agents/code-reviewer.md 를 만들어줘.  
name: code-reviewer / tools: Read, Grep, Glob /  
description "코드 리뷰할 때. 버그·보안을 심각도순"
```

>_ 일반 터미널 – 파일 확인

```
$ ls .claude/agents/  
  
code-reviewer.md  
← 파일이 있으면 자동 등록됨
```

</> ② 직접 작성 – code-reviewer.md 전문

```
---  
name: code-reviewer  
description: 코드 변경을 리뷰할 때 사용.  
            버그·보안·컨벤션 위반을 심각도순 보고.  
tools: Read, Grep, Glob # 읽기 전용  
---  
  
너는 시니어 리뷰어다. 심각도(높음/중간/낮음)  
별로 보고하고, 문제 없으면 "이상 없음"이라 하라.  
tools에 수정 도구가 없으니 제안까지만 하라.
```



핵심 서브에이전트는 .claude/agents/ 에 파일만 있으면 자동 등록된다. 파일을 만든 뒤 claude 를 새로 켜고, (2)에서 실제로 호출해 발동을 확인한다.

발동하는가 – 탐색은 서버에서, 메인엔 요약만

</> claude – 이름을 박아 호출

```
> code-reviewer 서버에이전트로  
  crawler/fetch.py 변경을 리뷰해줘.
```

이름 없이 안 뜨면?

"code-reviewer 서버에이전트로" 처럼 이름을 박으면 확실하다. 자동 발동은 description 을 트리거가 잘 걸리게 써야 한다(스킬과 같은 기술).

> 발동 – 서버 내부는 접히고 요약만

code-reviewer(Review fetch.py)

L Read·Grep·Glob ... (서브 컨텍스트)

L Done

[Critical] URL 스킴 미검증 –

urlopen 이 file:///·ftp:/// 도 열어 SSRF

[High] bare except·timeout 없음·FD 누수



컨텍스트 절약 가운데 Read·Grep 수집 줄이 메인 대화에 안 쌓인다 – 서버 컨텍스트에서 소모되고 메인엔 결론만 남는다(7.2가 눈에 보이는 순간).

권한은 이름이 아니라 정의 파일이 정한다

</> (3) 읽기 전용 검증 – 직접 고치려면?

> 방금 지정한 것들을 `code-reviewer`가
직접 고쳐줘.

>_ `code-reviewer` – 못 고친다

읽기 전용(`Read·Grep·Glob`)이라

파일을 수정할 수 없습니다.

수정안을 메인에서 적용할까요?

</> (4) 변형 – Write 를 준 `test-writer`

```
tools: Read, Write, Grep, Glob
# Write 한 줄 추가가 유일한 차이
```

>_ `test-writer` – 파일을 실제로 만든다

```
$ ls tests/
```

```
conftest.py  test_fetch.py
```

← 리뷰어는 못 했던 일(`Write` 권한)



체크포인트 같은 "서브에이전트"인데 `tools` 한 줄 차이로 하나는 읽기만, 하나는 파일을 쓴다 – "실수로 고치는 리뷰어"가 구조적으로 불가능(7.3).  [스크린샷]  `code-reviewer` 발동 화면.

위임의 층위 + Agent Teams

층위	무엇	자율성	언제
Subagents	한 세션 안의 하위 위임 (오늘)	낮음	탐색·리뷰·독립 하위 작업
Agent Teams	공유 태스크·메일박스로 에이전트끼리 협업	중간	반복 협업(구현+리뷰+테스트)
Dynamic Workflows	코드로 짜는 오케스트레이션(fan-out·pipeline)	높음	대규모·조건부 흐름 자동화

Agent Teams 공유 태스크 리스트('누가 뭘 하는 중/끝냈나')와 메일박스(에이전트 A→B '인터페이스 바뀌었으니 확인해', 메인 을 안 거치는 가로 소통)로 N:N 자율 협업 — 사람 팀의 칸반 + 메신저.

⚠ 리서치 프리뷰
실험적 — API·동작이 바뀔 수 있다. 지금은 개념으로, 자동화는 안정화 후.

 **병렬 위임** 의존 없는 하위 작업은 서브 여러 개를 동시에 — 메인 이 오케스트레이터로 분배→수집→종합. Worktree가 사람이 분배하는 병렬이라면, 이건 에이전트가 분배하는 병렬.

켜는 법 + 업계 지형 — OpenClaw · Hermes

</> settings.json — 활성화 (기본 꺼짐)

```
// .claude/settings.json (또는 ~/.claude/)
{
  "env": {
    "CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS": "1"
  }
}
```

안 켜면 안 뜬다

`claude update`가 "up to date"여도 자동으로 안 켜진다. 안 켜 채 "팀 만들어줘"라고 해도 그냥 **Subagent**로 빠진다. 켜 뒤엔 자연어로 팀원을 띄운다.

업계 지형 — 왜 내장을 쓰나

OpenClaw ≈ **Agent Teams** 메시지·공유 태스크 (N:N)

Hermes ≈ **Subagents** 부모에게만 보고 (1:N)

⚠ 2026-04 Anthropic이 구독(Pro/Max) 연동 차단 — 코드·커밋에서 OpenClaw·hermes.md 탐지 시 API 요금제로 전환.
그래서 **내장 Subagents / Agent Teams**가 안전한 표준.



대체의 한계 협업 구조(팀·서브)는 내장으로 그대로 얻지만, 각 제품 고유의 스킬 자동학습·거대 스킬 생태계까지 따라오지는 않는다.

병렬 3축 – Worktree vs Subagents vs Agent Teams

기준	Git Worktree (6교시)	Subagents (7교시)	Agent Teams (프리뷰)
격리 수준	프로세스(폴더·브랜치·세션)	컨텍스트(같은 폴더)	컨텍스트 + 공유 상태
파일 충돌	구조적으로 불가(폴더 분리)	가능 – 동시 수정 부적합	공유 리스트로 조율
맞는 작업	컴포넌트 병렬 구현(쓰기)	탐색·리뷰·분석(읽기)	반복 협업(구현+리뷰)
조율 주체	사람(탭 전환·병합)	메인 에이전트(분배→종합)	팀 자신(자율 조율)

경험칙 쓰기가 겹치면 Worktree · 읽기 위주 위임이면 Subagent · 반복되는 협업이면 Agent Teams(실험적). 셋은 경쟁이 아니라 조합.

 **7교시 정리** Subagent = 독립 컨텍스트·독립 권한의 위임(.claude/agents/, description=트리거). Agent Teams까지 더하면 병렬은 3축 – 공통 원리는 하나: 독립 컨텍스트에 일을 나누고 결과를 모은다. 다음(8교시): 뼈대에 진짜 DB를.

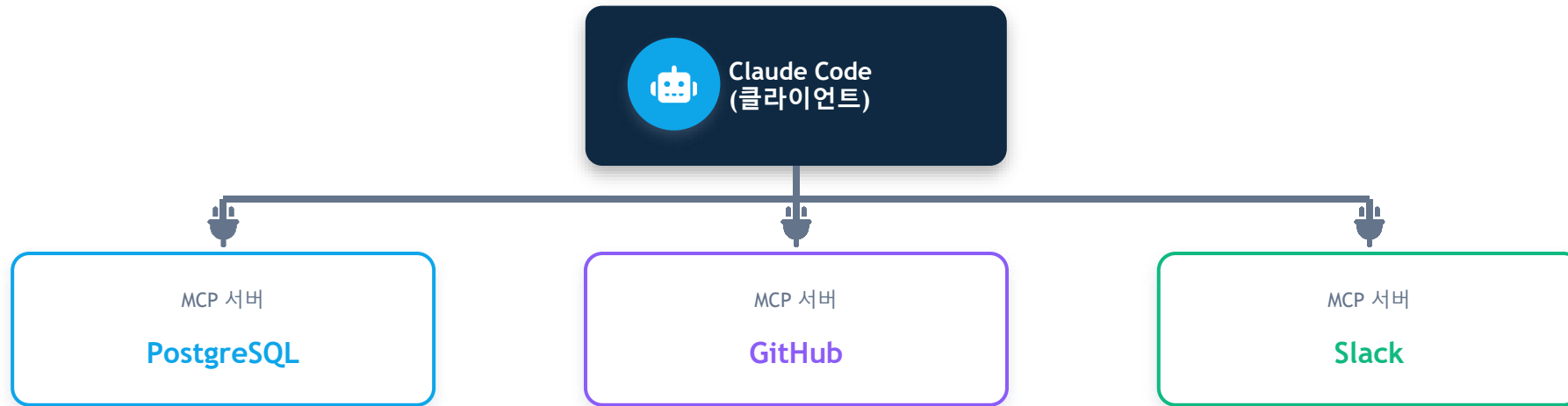


PERIOD 8 · 실습 ★★

MCP 연동 + DB

지금까지 에이전트의 손은 파일과 셀에만 닿았다. MCP(Model Context Protocol)는 에이전트를 외부 시스템(DB·API·SaaS)에 잇는 표준 규격이다 — 도구별 전용 연동을 N×M으로 만드는 대신, 서버 하나를 꽂으면 MCP를 아는 모든 에이전트가 쓴다. 그래서 'AI의 USB-C'라 불린다.

MCP – 에이전트의 USB-C



구조 [Claude(클라이언트)] ↔ [MCP 서버] ↔ [실제 시스템]. 서버가 도구 목록(query·list_tables...)을 노출하면, 에이전트는 그 도구를 에이전틱 루프 안에서 골라 쓴다(Day 1 1.3 루프에 도구가 추가될 뿐).



가운데 Claude, 좌우에 MCP 서버들(PostgreSQL·GitHub·Slack)이 같은 규격 플러그로 꽂힌다
— N×M 전용 연동 대신 서버 하나. 우리가 할 일은 연결과 권한 설정.

claude mcp add + .mcp.json 스코프

</> 서버 연결 (bash)

```
# PostgreSQL MCP 서버 연결
claude mcp add postgres -- npx -y \
  @modelcontextprotocol/server-postgres \
  "postgresql://app:pass@localhost:5432/vibe"
claude mcp list          # 등록 확인
# 세션 안에서는 /mcp 로 상태·도구 확인
```

.mcp.json 스코프 – 설정 파일과 같은 철학

local (기본)

개인·이 프로젝트 한정 – 나만 쓰는 실험적 연결

project

.mcp.json(커밋해 팀 공유) – 팀 표준 연결(우리 DB)

user

개인·모든 프로젝트 – 어디서나 쓰는 개인 도구



팀 공유 claude mcp add --scope project → .mcp.json이 저장소에 남아 harness의 일부가 된다(접속 문자열의 시크릿은 환경변수 참조로).  [스크린샷] /mcp 상태 화면.

ERD → 실제 스키마 – 3교시의 그림이 실물이 된다

> PostgreSQL 준비 + 스키마 생성

```
# 로컬 실습 DB (운영 DB 아님 - 부록 B)
$ docker run -d --name vibe-pg \
  -e POSTGRES_PASSWORD=pass \
  -e POSTGRES_DB=vibe \
  -p 5434:5432 postgres:16-alpine
```

> docs/ERD.md·store/schema.sql 대로
테이블을 만들고, 목록·칼럼을 조회해
ERD와 일치하는지 보여줘 → MCP가 실행

</> store/schema.sql (day2/08_postgres_db.ipynb)

```
CREATE TABLE items (
  id          SERIAL PRIMARY KEY,
  source      TEXT NOT NULL,
  url         TEXT NOT NULL UNIQUE,  -- 중복 방지
  title       TEXT NOT NULL,
  content     TEXT,
  created_at  TIMESTAMP DEFAULT now() );

CREATE TABLE summaries (
  id SERIAL PRIMARY KEY, target_date DATE,
  body TEXT, sent_at TIMESTAMP );  -- NULL=미발송
```



3단 일치 생성된 스키마가 ERD와 1:1인가(UNIQUE 제약까지) — 문서→코드→실물의 3단 일치. 보조 노트북 day2/08_postgres_db.ipynb로 그 아래층(파이썬+SQL)을 한 번 직접.

자연어 쿼리·마이그레이션 + DB 보안



[스크린샷] 자연어 → SQL (8.7)

"소스별로 집계해줘" 요청에 에이전트가 SQL 생성 → 실행 → 표로 답하는 화면.
스키마를 아는 에이전트는 SQL 비서"

마이그레이션 "items에 요약 완료 여부 칼럼이 필요해. 변경안을 Plan Mode로." → 계획 검토 → 적용 → ERD 문서도 함께 갱신(문서와 실물의 동기화 — 살아있는 문서).



DB 권한은 좁게 (최소 권한)

- 실습·개발엔 격리된 DB(로컬/컨테이너), 운영 직결 금지
- 읽기만 필요하면 읽기 전용 계정 — 계정이 곧 에이전트의 권한 상한
- 파괴적 쿼리(DROP·DELETE)는 deny·혹·Plan Mode로 겹겹이



권한의 연장 에이전트 손에 DB가 들어오는 순간 위험도 함께 온다 — MCP 서버에 주는 계정이 곧 권한 상한(Day 1 5교시 최소 권한의 연장). 인젝션은 Day 3 보안에서 심화.

Day 2 마무리 + Day 3 예고

오늘의 궤적

자원 관리	스펙(PRD→분해→지시)
설계문서 5종·스캐폴딩	기존 코드(E→P→C→C)
검증 루프(게이트·시각)	Worktree·Subagents 병렬
MCP + DB	

```
git commit -m "Day2: 설계문서·검증 루프·병렬·DB 연결"
```



Day 3 – 자동화·통합·품질

관통 프로젝트는 이제 문서로 고정된 설계 + 테스트가 지키는 뼈대 + 진짜 DB를 가졌다.

크롤링 크롤러를 진짜로 만들고

보안 인젝션·시크릿 방어

품질 리뷰·CI로 품질을 지키고

완성 Discord 알림 + harness 팀 표준화

명세가 명확할수록 위임할 수 있다 글(스펙·설계·테스트)이 정확해진 만큼 병렬(Worktree·Subagents·Agent Teams)이 가능해졌다 — 둘은 같은 능력의 양면.

핵심 정리 – 8교시를 한 줄씩

1 컨텍스트·비용 — /cost·/compact, 모델 티어링·effort, CLAUDE.md 다이어트

2 스펙 주도 — 4요소 지시 + PRD→태스크 분해→구현 지시 파이프라인

3 5대 설계문서(유스케이스~ERD) + 교차 검수 + 문서→코드 스캐폴딩

4 기존 코드베이스 — Explore→Plan→Code→Commit + 특성화 테스트·디버깅

5 검증 루프 — RED→GREEN·cheating 방지·테스트 게이트 혹·시각 피드백

6 Git Worktree — 프로세스 병렬(폴더·브랜치·세션 분리)·분할 원칙·병합

7 Subagents(.claude/agents/)·Agent Teams — 병렬 3축

8 MCP + PostgreSQL — ERD가 실제 스키마로·자연어 쿼리·DB 보안

명세가 명확할수록 위임할 수 있다 — 글로 정의하고, 남의 코드를 읽고, 병렬로 부려라.

부록 – 실습 구성 · 트러블슈팅 · 키워드

>_ 실습 구성(인라인) – 교시별 산출물

지시서 md 없음 – 각 교시 [실습] 인라인
(데모→따라→적용) + 8교시 보조 ipynb

1교시 /cost·/compact + CLAUDE.md 다이어트
2교시 좋은 vs 나쁜 → PRD·태스크 분해
3교시 설계문서 5종 + Mermaid + 스캐폴딩
4교시 E→P→C→C + 특성화 테스트·리팩토링
5교시 RED→GREEN + 게이트 혹 + 시각 피드백
6교시 2-worktree 병렬 + 병합
7교시 커스텀 서브에이전트 + 병렬 3축
8교시 MCP → ERD 스키마 → 자연어 쿼리
+ day2/08_postgres_db.ipynb (원리)

필요 도구: Claude CLI·git·에디터 + PostgreSQL



트러블슈팅

MCP 서버 안 붙음 — claude mcp list·/mcp 확인, 접속 문자열 오타가 최다

/compact 후 맥락 소실 — 중요 결정은 압축 전 CLAUDE.md/메모리로

worktree 충돌/미로 — git worktree list로 현황, 끝난 건 즉시 remove

서브에이전트 발동 안 함 — description을 '언제 쓰는지'로 구체화

AI가 테스트 고침(cheating) — 수정 금지+diff 확인+PreToolUse 혹

에이전트가 DB 과감 — 계정 권한 좁게(읽기 전용·격리 DB·deny)



핵심 키워드

컨텍스트·비용 · /cost·/compact · 티어링·effort · PRD·태스크 분해 · 4요소 지시 · 5대 설계문서 · 유스케이스·컴포넌트·인터페이스·시퀀스·ERD · Mermaid · 스캐폴딩 · Explore→Plan→Code→Commit · 특성화 테스트 · RED→GREEN · 테스트 게이트 혹 · 시각 피드백 · git worktree · Subagents · Agent Teams · 병렬 3축 · MCP · .mcp.json · PostgreSQL